

From a weighted sum to a neuron

A neural network becomes expressive when affine maps are separated by nonlinear activations.

The loss says what counts as a good prediction. The network says which predictions are possible.

1 • The linear starting point

One scalar output begins with

$$z = \mathbf{w}^\top \mathbf{x} + b.$$

Each edge contributes “weight $\times$ incoming value”; the node adds the contributions and the bias.

INPUT	$\mathbf{x} \in \mathbb{R}^d$
WEIGHTS	$\mathbf{w} \in \mathbb{R}^d$
BIAS	$b \in \mathbb{R}$
OUTPUT	$z \in \mathbb{R}$

Concrete weighted sum

For $\mathbf{x} = (2, -1)$, $\mathbf{w} = (1.5, 0.5)$, and $b = -0.2$:

$$z = 1.5(2) + 0.5(-1) - 0.2 = 2.3.$$

The bias is equivalent to an extra constant feature equal to 1.

2 • Batch form and shape audit

Stack n examples as rows of $\mathbf{X} \in \mathbb{R}^{n \times d}$:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b\mathbf{1}.$$

Or append a ones column $\tilde{\mathbf{X}}$ and stack the bias into $\boldsymbol{\theta}$:

$$\hat{\mathbf{y}} = \tilde{\mathbf{X}}\boldsymbol{\theta}.$$

Shape check: $(n \times (d+1))((d+1) \times 1) \rightarrow (n \times 1)$.

3 • A neuron adds a nonlinear stage

$$z = \mathbf{w}^\top \mathbf{x} + b, \quad a = \varphi(z).$$

The affine stage chooses an input direction and offset. The activation gates or reshapes the resulting score.

For $\mathbf{x} = (2, -1)$, $\mathbf{w} = (1, -1)$, $b = 0$:

$$z = 3, \quad a = \text{ReLU}(3) = 3.$$

4 • Activation map

Name	Rule / range	Use and warning
sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	probability-like switch; saturates
tanh	$\tanh(z) \in (-1, 1)$	signed switch; saturates
ReLU	$\max(0, z)$	simple ramp; dead for $z < 0$
GELU	$\approx z\sigma(1.702z)$	smooth, input-dependent gate

Saturation is a learning issue

A flat activation has a tiny derivative. For sigmoid,

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Near $z = 0$, $\sigma'(0) = 0.25$; near $z = 6$, $\sigma'(6) \approx 0.0025$. The same input nudge produces about $100 \times$ less output change in the saturated regime.

5 • Why nonlinearity is essential

Two affine layers collapse:

$$\mathbf{h} = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1, \quad \mathbf{y} = \mathbf{W}_2 \mathbf{h} + \mathbf{b}_2$$

$$\mathbf{y} = \underbrace{\mathbf{W}_2 \mathbf{W}_1}_{\mathbf{W}_{\text{eq}}} \mathbf{x} + \underbrace{\mathbf{W}_2 \mathbf{b}_1 + \mathbf{b}_2}_{\mathbf{b}_{\text{eq}}}.$$

Without φ , extra layers add parameters but no new bends or learned feature shapes.

Affine map = select and shift a direction.
Activation = introduce a bend. Composition = build a representation.

Read an MLP by its shapes and features

Hidden units learn coordinates in which the final prediction can be simpler.

1 • One hidden layer

$$h = \varphi(W_1 x + b_1), \quad z = W_2 h + b_2.$$

With input width d , hidden width m , and K outputs:

$$\begin{aligned} W_1 &\in \mathbb{R}^{m \times d} \\ W_2 &\in \mathbb{R}^{K \times m} \end{aligned}$$

$$\begin{aligned} b_1 &\in \mathbb{R}^m \\ b_2 &\in \mathbb{R}^K \end{aligned}$$

$$(m \times d)(d \times 1) \rightarrow m, \quad (K \times m)m \rightarrow K.$$

$$\text{Trainable scalars} = m(d + 1) + K(m + 1).$$

2 • XOR: features can solve what one line cannot

A single binary linear score has one straight decision boundary. For the four XOR corners, create

$$h_1 = \text{ReLU}(x_1 + x_2), \quad h_2 = \text{ReLU}(x_1 + x_2 - 1),$$

then use

$$z = 2h_1 - 4h_2 - 1, \quad \hat{y} = \mathbb{1}[z \geq 0].$$

$$(0, 0) \rightarrow 0 \quad (1, 0) \rightarrow 1 \quad (0, 1) \rightarrow 1 \quad (1, 1) \rightarrow 0$$

Exact on the four Boolean points does not imply the same rule labels every point inside filled XOR regions. State the input domain.

3 • Width and depth answer different needs

Width: more units in one layer; more features at the same stage.

Depth: more successive transformations; later features reuse earlier features.

Choose width for variety and depth for composition. A layer receives the previous representation, not the original input unless an explicit skip connection supplies it.

4 • ReLU networks assemble hinges

One shifted ReLU is a hinge:

$$h_{j(x)} = \text{ReLU}(w_j x + b_j).$$

A linear output combines many such features:

$$g(x) = c + \sum_j v_j \text{ReLU}(w_j x + b_j).$$

The input weights and biases place the hinges; the output weights choose their slope changes. In higher dimensions, ReLU units fold space along hyperplanes.

5 • Universal approximation: read the claim carefully

For a continuous target f on a closed, bounded region D and any $\varepsilon > 0$, a sufficiently wide finite network with a suitable nonlinear activation can satisfy

$$\sup_{x \in D} |f(x) - g(x)| < \varepsilon.$$

This is an **existence and capacity** statement. It does not promise that gradient-based training will find the network, that the network will be small, or that it will generalize beyond D .

6 • Output-layer contract

Task	Raw output	Final map
regression	one or more scores	identity / task
binary	one logit z	constraint
K classes	K logits	sigmoid $\sigma(z)$ softmax over K

The hidden representation may stay the same; the number and interpretation of output nodes follows the target.

Fast architecture audit

- ✓ Write the shape of every value and parameter.
- ✓ Point to the nonlinearity between affine layers.
- ✓ Explain each hidden layer as a learned feature map.
- ✓ Match the output width and transform to the task.
- ✓ Separate representability from trainability.